

```
@page { margin: 14mm 12mm; } body { font-family: "DejaVu Sans",
Arial, sans-serif; font-size: 10.5pt; line-height: 1.34; color: #1a2733;
max-width: 100%; margin: 0; } h1 { font-size: 19pt; color: #12324b;
border-bottom: 2px solid #12324b; padding-bottom: 3px; } h2 { font-
size: 14pt; color: #12324b; border-left: 4px solid #12324b; padding-
left: 7px; margin-top: 16px; } h3 { font-size: 12pt; color: #234;
margin-top: 12px; } table { border-collapse: collapse; width: 100%;
margin: 8px 0; font-size: 9pt; } th, td { border: 1px solid #b7c2cc;
padding: 3px 6px; text-align: left; vertical-align: top; } th {
background: #12324b; color: #fff; } tr:nth-child(even) td {
background: #f2f5f7; } code { background: #eef1f4; padding: 1px
3px; border-radius: 3px; font-size: 8.8pt; } pre { background:
#0f1b26; color: #dfe7ee; padding: 8px 10px; border-radius: 5px; font-
size: 8.4pt; overflow-x: auto; white-space: pre-wrap; } pre code {
background: transparent; color: inherit; }
```

INTEGRATION_GUIDE

O-RAN VIP Suite — Integration Guide

Author: AVIK MAJUMDAR **Project:** AVIK VIP FACTORY — O-RAN VIP Suite **Document:** Integration Guide **Revision:** 1.0 **Date:** 2026-08-09

1. Introduction

This Integration Guide describes how to bring up the toolchain, build and run every lane on every supported simulator, generate code and functional coverage, dump waveforms, add new components, integrate a licensed simulator, wire the suite into CI, and troubleshoot the known UVM-on-Verilator issues. It targets an engineer integrating the suite into a project or continuous-integration flow.

2. Directory Structure

```
oran_vip_suite/
  Makefile                all-simulator
build/run/regress/coverage/waves
  compile.sh              cross-simulator compile wrapper
  README.md               quick start
  MANIFEST.md5            integrity manifest
  gen/                    Lane-1 framework
  gen_stim.py             Z3 stimulus generator (pos / --neg / --
directed)
  adjudicate.py           result.json truth authority
(PASS_FAIL_POLICY)
  build_suite.py          SPECS + SV emitters + coverage-enabled build
(GATE 2/3)
  run_gates.py            GATE 4-8 driver
(directed/random/negative/coverage)
  code_cov.py             GATE-8 code coverage (line/branch/toggle +
waivers)
  corner.py               corner/edge cases
  report.py               full regression + coverage report +
dashboard
  stamp_headers.py        authored file-header stamper
  common/pkg/             shared eCPRI field model (pathfinder)
  components/<slug>/      Lane-1 per component
    rtl/<slug>_pkg.sv      field model: struct, pack/unpack, is_legal,
first_violation
```

```

rtl/<slug>_codec.sv    combinational codec (block under test)
tb/<slug>_tb_top.sv    self-checking testbench
sim_main.cpp          Verilator main + coverage flush
spec.json             single source of truth
(fields/legal/cross/cover)
  run.sh               per-component compile+run+adjudicate
  docs/GATE0_*, GATE1_* vplan + architecture
  sim_out/             build + result.json
lane2/common/muvvm_pkg.sv    UVM-1.1-compatible subset (Lane-2A)
lane2/gen/gen_uvm.py         Lane-2A env generator
lane2/gen/gen_uvm_native.py  Lane-2B native UVM generator
lane2/components/<slug>/     Lane-2A UVM-subset envs
lane2/lane2b_native/
  ecpri_transport/         native UVM reference + SVA
interface + run_lane2b.sh
  suite/                   13-component combined native UVM +
build_suite_uvm.sh + run_all.sh
  lane2/uvm050/            Verilator 5.050 + UVM-on-Verilator recipe
  build_verilator050.sh    source build of Verilator 5.050
  setup_uvm050.sh          clone + patch Accellera uvm-core
(DPI shim)
  vpi_stub.cc              vpi_get_vlog_info stub
  regression/              suite_regression_full.json,
combined_regression.json,
                           gate4_8_report.json, code_cov.json,
lane2a_report.json, lane2b_report.json
  docs/                    guide, architecture, VRD, FD, this guide,
cross-sim README,
                           coverage report + dashboard, pdf/
  signoff/                 ORAN_VIP_SUITE_SIGNED_OFF.md

```

3. Prerequisites

- Linux (validated on Ubuntu 24.04), 2+ cores, ~5 GB free disk.
- Python 3.10+ with z3-solver (pip install z3-solver).
- Verilator 5.020 (apt) for Lane-1 / Lane-2A; Verilator 5.050 (source-built) for Lane-2B.
- Accellera uvm-core (cloned + patched by setup_uvm050.sh) for Lane-2B.
- Optional: a commercial UVM simulator (VCS / Questa / Xcelium) for Lane-2B portability.
- pandoc + wkhtmltopdf for document PDF generation.

4. Toolchain Bring-Up

4.1 Verilator 5.050 (mandated primary)

```

make verilator050
# builds git tag v5.050 -> /usr/local/bin/verilator (~25 min on 2
cores)
verilator --version          # Verilator 5.050 2026-07-01 rev v5.050

```

4.2 UVM on Verilator 5.050

```

make uvm-setup              # clones Accellera uvm-core and applies
the DPI shim
export UVM_HOME=/tmp/uvmtest/uvm

```

The shim (Section 11) is what allows full Accellera UVM to elaborate and run on Verilator.

5. Build and Run — Makefile Reference

Variables: SIM (verilator5020 | verilator5050 | vcs | questa | xcelium), COMPONENT (<slug> | all), TEST, SEED, N (txns/test), RAND (random seeds), WAVES (0|1), UVM_HOME.

Target	Action
make help	list targets
make lane1	Lane-1 full: build + gates 2-8 + corner + coverage + report
make lane2a	Lane-2A UVM-subset (all or COMPONENT)
make lane2b SIM=verilator5050	Lane-2B native UVM: build once + run 13
make lane2b SIM=vcs\ questa\ xcelium	Lane-2B native UVM on a licensed sim
make regress RAND=100	ALL lanes → combined_regression.json
make regress-l1 COMPONENT=all	Lane-1 smoke (seeds 1,2,3)
make regress-l2\ l3\ l5	directed / random / full via run_gates
make compile COMPONENT=<slug>	compile one component (Lane-1)
make run COMPONENT=<slug> TEST=smoke SEED=1	single run
make coverage	code (stmt/branch/toggle) + functional reports + dashboard
make waves SIM=<sim> COMPONENT=<slug> TEST=<t> SEED=<n>	dump waveform
make clean	remove build dirs / stimulus / waves

./compile.sh <sim> <lane> [component] is the direct compile wrapper used by CI when a finer-grained step is needed.

6. Coverage Generation (CODE and FUNCTIONAL)

make coverage

- **Functional:** the monitor emits COVROW lines; report.py merges COV-### bins across smoke + random + directed + corner into regression/suite_regression_full.json (func_cov per component) and docs/FULL_COVERAGE_REPORT.md + docs/COVERAGE_DASHBOARD.html. Native covergroup (Lane-2B) is sampled in-simulator.
- **Code:** code_cov.py compiles with --coverage, runs positive + negative + a corrupt-golden checker self-test, merges coverage.dat, and writes regression/code_cov.json (line/branch/toggle; TB-defensive branch waivers documented).
- **Commercial simulators:** VCS -cm line+cond+fsm+branch+tg1+assert then urg; Questa +cover -coveropt 3 then vcover report; Xcelium -coverage all then imc.

7. Waveform Dumping (per simulator)

make waves SIM=verilator5050 COMPONENT=uplane TEST=smoke SEED=1 #
FST -> gtkwave

- **Verilator:** --trace-fst (WAVES=1 in run.sh); output *.fst, view with gtkwave.
- **VCS:** vcs -debug_access+all -kdb; at runtime UCLI dump -add / ; run → VPD/FSDB.
- **Questa:** vsim -voptargs=+acc -do 'add wave -r /*; run -all' → WLF (wlf2vcd to convert).
- **Xcelium:** xrun -access +rwc -input '@database -open waves -shm; probe -create -all -depth all; run' → SHM/VCD. Waveforms

are dumped on FAIL by policy and preserved (never deleted).

8. Adding a New Component

1. Add one entry to `gen/build_suite.py` SPECS: fields (≤ 64 bits total, auto-padded), legal per field, cross constraints, cover kinds.
2. `make lane1` — regenerates and builds the Lane-1 TB, runs gates.
3. Lane-2A and Lane-2B envs are generated from the same `spec.json` by `lane2/gen/gen_uvm.py` and `lane2/gen/gen_uvm_native.py` — no hand-editing.
4. `python3 gen/stamp_headers.py` re-applies authored headers to generated files.

9. Licensed-Simulator Integration (Lane-2B)

The native UVM source is standard Accellera UVM (no Verilator-specific code). Filelist: `lane2/lane2b_native/ecpri_transport/ecpri.f`. Set `UVM_HOME`. Commands:

```
vcs -sverilog -full64 -ntb_opts uvm-1.2 -f ecpri.f -cm
line+cond+fsm+branch+tgl+assert
qverilog -sv -mfcu +incdir+$UVM_HOME/src $UVM_HOME/src/uvm_pkg.sv -f
ecpri.f +cover
xrun -uvm -f ecpri.f -coverage all
```

`make lane2b SIM=vcs|questa|xcelsius` invokes `run_lane2b.sh` with the correct switches. The generator emits all thirteen components identically; only the top filelist changes.

9a. RAL (uvm_reg) Register Model

The register abstraction layer lives in `lane2/ral/` (`oran_ral_pkg.sv`, `oran_ral_tb_top.sv`, `build_ral.sh`). It is native Accellera UVM and builds with the same DPI shim as Lane-2B.

```
make ral SIM=verilator5050          # build + run oran_ral_test ->
PASS (163 ops, 0 err)
# direct:
cd lane2/ral && UVM_HOME=/tmp/uvmtest/uvm bash build_ral.sh
./obj_ral/simv_ral +UVM_TESTNAME=oran_ral_test +ntb_random_seed=1
```

On VCS/Quarta/Xcelsius, compile `oran_ral_pkg.sv` + `oran_ral_tb_top.sv` against `$UVM_HOME` and run `+UVM_TESTNAME=oran_ral_test` — identical source, no simulator-specific code. The run emits `RAL_SUMMARY,ops=<n>,errs=<n>` and `** TEST PASSED **`; the adjudication marker is the UVM Report Summary (`UVM_ERROR : 0, UVM_FATAL : 0`). Result: `lane2/ral/ral_result.json`.

10. Continuous Integration

Recommended GitHub Actions layout: - **Stage 1 (always)**: apt Verilator 5.020 → `make lane1` + `make lane2a` → adjudicate. - **Stage 2 (self-hosted or licensed runner)**: Verilator 5.050 (or commercial) → `make lane2b SIM=verilator5050` + `make ral SIM=verilator5050` → adjudicate. - **Artifacts**: `regression/*.json`, `docs/FULL_COVERAGE_REPORT.md`, `docs/COVERAGE_DASHBOARD.html`, `signoff/ORAN_VIP_SUITE_SIGNED_OFF.md`. Each stage fails the build if any `result.json` STATUS is not PASS / EXPECTED_FAILURE_DETECTED.

11. Troubleshooting — UVM on Verilator 5.050

The full Accellera UVM library links on Verilator 5.050 with three adaptations, all persisted in `lane2/uvm050/` and applied by `setup_uvm050.sh`:

1. **HDL backdoor #error.** `uvm_hdl.c` errors when no vendor backend is defined. The register backdoor is unused, so it is replaced with no-op stubs. The stub signatures must match `uvm_dpi.h` (`char*`, `p_vpi_vecval`) — not Verilator’s generated `const char*/const svLogicVecVal*` — otherwise the compile raises a conflicting declaration. C linkage resolves the call by symbol name, so the ABI-compatible stubs link.
2. **VPI polling include.** `uvm_dpi.cc` unconditionally includes `uvm_hdl_polling.c`, which needs full VPI; it is disabled (backdoor unused).
3. **vpi_get_vlog_info.** UVM queries the tool name via this VPI call; a small `vpi_stub.cc` provides it (product “Verilator”, version “5.050”).

Common runtime pitfalls and fixes: - **NOCOMP “No components instantiated”**: package test classes are not registered because Verilator elides `void'(T::type_id::get())`. Force registration with an observable side effect: `if (T::type_id::get() != null) reg_cnt++;` in the top, and pass the name explicitly: `if ($value$plusargs("UVM_TESTNAME=%s", tn)) run_test(tn);`. - **Simulation hangs**: a rand sequence-length field randomizes to billions of items. Make the item count non-rand and bounded; add a watchdog `#T`; `$finish` for the free-running clock. - **Build time**: the combined 13-component UVM build is compute-bound on 2 cores; ccache is enabled (`CCACHE_DIR`, `CCACHE_MAXSIZE=5G`) so re-links are fast.

12. Evidence and Signoff

`regression/combined_regression.json` is the top-level ledger (1522/1522 = 100%). `signoff/ORAN_VIP_SUITE_SIGNED_OFF.md` carries the per-component gate table and portability matrix. Every source file is authored (AVIK MAJUMDAR) and header-documented. Reproduce end to end: `make verilator050 && make uvm-setup && make regress RAND=100 && make coverage`.